

Database Management Systems

Address Data Path - System R

for pipelining
on single processor

→ Assess methods

→ Sorts (only left-deep plans)

→ Seq. scan

→ Nested Loop Soln (NL3)

→ Index scan (unclustered/clustered)

→ bit-merge sort

predicts no. of tables

→ $\text{Lat} = P_{\text{fetch}} + P_{\text{SS calls}} * w$

→ Statistical estimates :

→ NCARD: No. of tuple in a table

→ TCARD: "pgs. occupied by a table"

→ I(ARD): " " distinct reqs in a index

→ Uses DP. It's still exponential though.

Limitations :

→ Retention Factor calculation:

→ considers left-deep plans only

→ Makes major statistical assumptions

→ Do not attempt to "flatten" nested queries.

Prekrate	R F	Default
$wl = \text{value}$	$\frac{1}{N_{\text{keys}}(I)}$	$\frac{1}{10}$
$wl_1 = wl_2$	$\frac{1}{\max(N_{\text{keys}}(I_1), N_{\text{keys}}(I_2))}$	$\frac{1}{10}$
$wl > \text{value}$	$\frac{\text{High}(I) - \text{value}}{\text{High}(I) - \text{low}(I)}$	$\frac{1}{3}$
$\text{Min} < wl < \text{max}$	$\frac{\text{max} - \text{min}}{\text{High}(I) - \text{low}(I)}$	$\frac{1}{4}$

$$\begin{aligned} \text{Total no. of join trees} &= (n \cdot n!) \\ &= \frac{(2(n-1))!}{(n-1)!} \end{aligned}$$

→ TC can be reduced to $O(3^n)$ using DP. can be reduced to $O(n \cdot 2^n)$ considering left-deep plans only.

→ For test products we delayed to the end.

→ In each iteration, we return:

→ Optimal plan

→ "Interesting" plans

→ Cost estimates:

	Last Estimate
Unclustered index on PK = val	Height of B ⁺ tree + 1
Clustered index I matching ≥ 1 predicates	$\left(\text{No. of Pgs. of I} + \text{No. of Pgs. of R} \right) \times RFS$
Unclustered index I matching ≥ 1 predicates	$\left(\text{No. of Pgs. of I} + \text{No. of tuples of R} \right) \times RFS$
No index (seq. scan)	No. of pgs. of R

Adaptive Plan Diagrams

→ Plan Diagram Reduction (last-greedy algo)

- Assign a bin to each plan P
- Start @ top right corner & proceed in row-major order
- Put a copy of each query point into all bins s.t. a constraint is maintained.
- The plan bucket with the most no. of points shall 'swallow' all points in its bucket.

l=20f reduces virtually any plan diagram to ≤ 10 plans

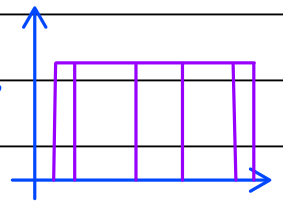
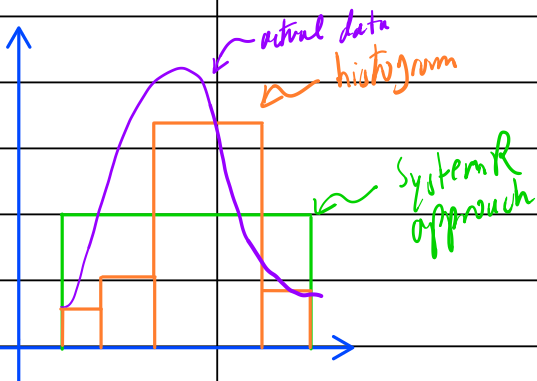
Histograms

→ Equi depth histograms

→ Data needs to be pre-sorted / pre-indexed.

We can also do sampling

→ Relatively difficult to maintain



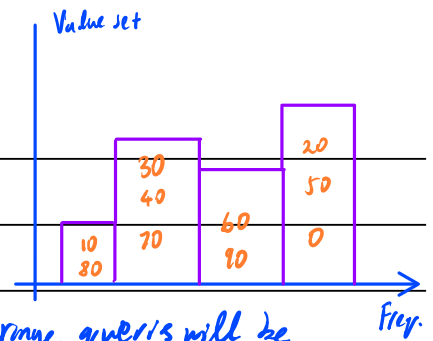
→ Serial histograms

→ Plotting value against frequency

→ Most optimal histogram to reduce error.

→ All domain values need to be represented; range queries will be difficult

→ Minimise $\sum n_i V_i$, where n_i = no. of values in bucket i , V_i is the variance in bucket i .



→ End-biased histogram

→ Serial histogram, but highest & lowest freq. values have their own buckets; remaining everyone has to share.

Sampling

→ Advantages:

→ Samples are row-wise, which helps in correlation.

→ Some aggregate functions can be answered using sample data itself.

→ Easier to compute

EXISTS

→ Disadvantages

→ Unbounded errors (may be too high rarely)

→ Biased against low freq. data.

→ May need to repeat for each query

→ Central limit theorem

→ Let true mean = μ & true variance = σ^2

→ In a sample (with n samples), sample mean = M & sample variance = S^2

→ The C.L.T. states that the error statistic, $\frac{M - \mu}{\sqrt{\frac{\sigma^2}{n}}}$ will have $N(0, 1)$ distribution, regardless of original data.

→ Then $\mu = M \pm t_{1-\alpha/2, n-1} \sqrt{\frac{\sigma^2}{n}}$

Student's t distribution
confidence
no. of samples - 1 (for deg. of freedom)

→ This implies that, with a confidence of $(1-\alpha)$, the true mean is within the limit of the sample mean. For $m > 30$, the distribution shall effectively be normal.

Algo: Using 'partition' approach

b is upper bound on no. of random samples
Accuracy indicator, d is more aggressive than ϵ

$s = 0$ // Samples
 $m = 0$ // No. of samples

while $(s < \kappa_1 b d (\lambda + 1))$ and $(m < \kappa_2 e^2)$ do:

$s += \text{Random Sample } ()$ // Returns no. of records in random sample
 $m++$

end

$\tilde{A} = ns/m$ // $\tilde{A}$ represents estimated cardinality of result

$$\kappa_1 = \left\lceil \phi^{-1} \left(\frac{1 + \sqrt{p}}{2} \right) \right\rceil^2$$

$$\kappa_2 = \left\lceil \phi^{-1} \left(\frac{1 + p}{2} \right) \right\rceil^2$$

* Because $0 \leq p \leq 1$,

$$\kappa_1 \geq \kappa_2$$

Guarantees of the algo:

→ $\tilde{A}$ will be within $\left(\frac{A}{d}, \frac{bn}{e} \right)$

→ Lemma 2.1

sufficient sample size

$$P \left(\sum_{i=0}^m x_i > \alpha \frac{V}{E} \right) = P \left[\frac{\sum_{i=1}^m x_i - mE}{\sqrt{mV}} > \frac{\alpha \frac{V}{E} - mE}{\sqrt{mV}} \right]$$

$$P_{\text{sufficient}} = P \left(\frac{V}{E^2} \right)$$

$$P_{\text{sufficient}} = \alpha \left(\frac{V}{E} \right)$$

$$= P \left[\frac{\sum_{i=1}^m x_i - mE}{\sqrt{mV}} > \alpha \frac{V}{E} \right]$$

$$= 1 - \phi \left(\frac{\alpha - \beta}{\sqrt{\beta}} \right)$$

$$P(\text{success}) = P\left(\frac{\text{success}}{\text{sufficient samples}}\right) * P(\text{sufficient samples})$$

$$= \left(2\phi\left(\frac{\sqrt{\beta}}{\alpha}\right) - 1\right) \left(\phi\left(\frac{\alpha - \beta}{\sqrt{\beta}}\right)\right)$$

→ Snowed data

→ $b \gg E$

10% of data produces 90% of output



* To prevent too sampling,
we can use sampling w/o
replacement. This is quite
expensive & hence never used

Assume we need a tuple that is "one in a million"

We shall have to sample $10^6 * n$, $d(d+1)$ tuples. Furthermore, there is no guarantee of finding the required sample as we may be trying the same sample repeatedly. This is why we use the second condn. in the loop

→ Implementation:

→ Selection

→ Join

→ Sampling can be
done on any index

→ Index can be on any attribute of source reln.

→ " nest " " join " " dest. "

outer
inner
(nested)

Plan Baynets

→ Runtime discovery of selectivities using compile-time selected budget of plans. This helps provide worst case guarantees.

→ Performance metric: $\text{Subopt}(q_e, q_a) = \frac{\text{cost}(P_{oe}, q_a)}{\text{cost}(P_{oa}, q_a)}$

estimated selectivity
actual selectivity
optimal plan @ estimated selectivity

optimal plan @ actual selectivity

2D Plan Bouquet

- Instead of iso cost lines, we use iso-cost planes. Each plane may have multiple plans.
- $\text{Cost} = \sum_{i=1}^K [A_i * \text{cost}(I_i)]$
 $\leq f \cdot \sum_{i=1}^K I_i$ ← Max. 4 from 1-unstable
 $\leq 4f$
- We can use more viz plan diagrams for reducing cost:
 $\text{Cost} \leq 4 * f_{\text{reduced}} * (1 + \lambda)$
- To further reduce the cost we can do selectivity monitoring. When executing a plan, even if it does not end up completing in the allotted 'budget', it may still provide some info. on the selectivity of the predicate. We can use this to further bound, and hence reduce the plan space.
- For D dimension, we can guarantee $O^2 + 3D$ max cost, (independent of the query)
- Considering uncertainty will further help in calculating plan diagrams @ runtime.
- We can also use Red-Black trees to further reduce plan space.

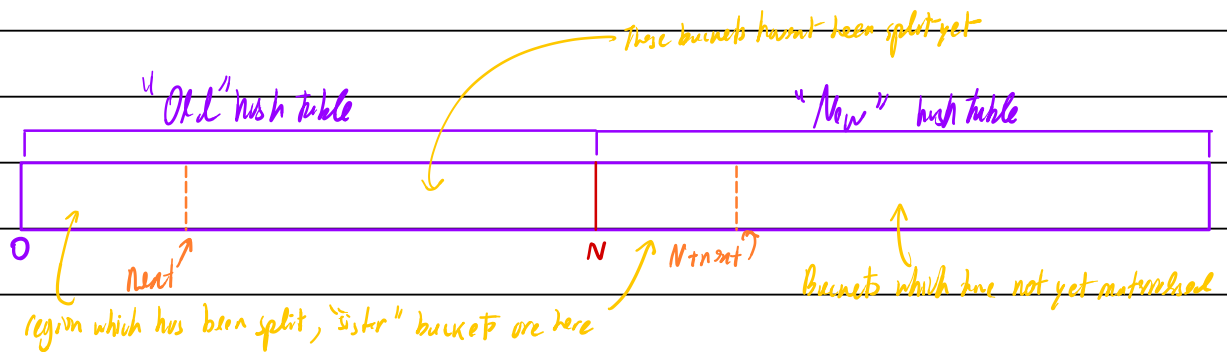
Linear Hashing

- Extensible hashing
 - When an overflow occurs, no. of buckets will be doubled, and the entire hashtable needs to be re-hashed.
 - Has potential to destroy the entire database by adding too many duplicates.
- Linear Hashing
 - Use a fty. of hash function: $h_i(x) = h(x) \cdot i \cdot (2^i \times N)$
 - Buckets are split in round-robin fashion
 - Insertion:
 - Method 1 (Two hash functions)
 - First apply old hash function, if the bucket is not split, add it into its respective bucket.
 - If the bucket has been split find out which of the two

split buckets the data item belongs to. Add it to the respective bucket.

→ Method 2 (Using one hash function & arithmetic)

→ Just apply the new hash function. If the function maps to a bucket which hasn't yet materialised, we obtain the original bucket using maths.



→ Duplicate entries still cause problems in access time, but the problem is localised (no collateral damage)

Bitmap Indices

→ Attribute value decomposition $C \leq \prod_{i=0}^n b_i$

→ We take $(n-1)$ values of b_i , b_n can be calculated as:

$$b_n = \left\lceil \frac{C}{\prod_{i=0}^{n-1} b_i} \right\rceil$$

→ Range encoding

→ Inequality operators can be written in terms of $\leq$

$$A < v \Rightarrow A \leq v-1$$

$$A > v \Rightarrow \neg (A \leq v)$$

$$A \geq v \Rightarrow \neg (A \leq v-1)$$

} Only works on discrete data

→ Space & Time complexity *due to implicit first coln.*

$$S_{\text{space}} = \sum_{i=1}^n (b_i - 1)$$

$$E_r = 2 \left(n - \sum_{i=1}^n \frac{1}{b_i} \right) - \left(1 - \frac{1}{b_i} \right)$$

$$E_c = 2 \left(n - \sum_{i=1}^n \frac{1}{b_i} \right)$$

$$\therefore E = \frac{2}{b} E_c + \frac{4}{b} E_r = 2 \left(n - \sum_{i=1}^n \frac{1}{b_i} \right) - \frac{4}{b} \left(1 - \frac{1}{b_i} \right)$$

→ Space-time trade off

	Bases	No. of components
Space optimality	$\langle b-1, \dots, b-1, b, \dots, b \rangle$	Increase no. of components
Time optimality	$\langle 2, 2, 2, \dots, \lceil \frac{C}{2^n-1} \rceil \rangle$	Decrease " " "

→ Space optimality: $\langle b-1, \dots, b-1, b, \dots, b \rangle$

→ r is the smallest integer that

satisfies: $(b-1)^{n-r} \cdot b^r \geq C$

	n	b
Space optimality	$\log_2 C$	2
Time optimality	1	C

$$\rightarrow \text{No. of bitmaps} = (n-r)(b-2) + r(b-1) = n(b-2) + r$$

→ Klee index: True 2-component index for max. optimality:

Bases: $\langle b_2 - \delta, b_1 + \delta \rangle$, where $b_1 = \sqrt{C}$, $b_2 = \lceil \frac{C}{\sqrt{C}} \rceil$, and

Used to maximise

time efficiency while

keeping space efficiency

time

$$\delta = \max \left\{ 0, \left\lfloor \frac{b_2 - b_1 + \sqrt{(b_2 - b_1)^2 - 4C}}{2} \right\rfloor \right\}$$

→ Bitmap storage (all true up some space)

→ Bitmap-level storage // best for time efficiency

→ Component " " // best for compression

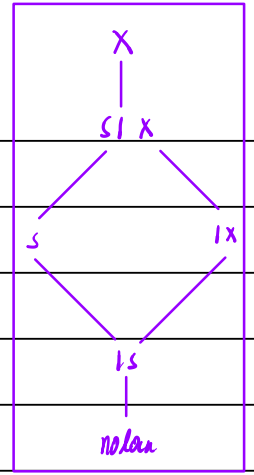
→ Index " "

* If no. of components are increased (space optimal) they will time cost of compression themselves is no need of explicit compression

All are stored in row-major order for compression optimality

Concurrency Control

- Create multiple locking units to maximise concurrency.
- Locking protocol
 - Lock all containing units with "intention locks" (IS/IX)
 - SIX: shared lock with intention to obtain exclusive lock
- Unlocking: Release all locks @ end of transactions
- Addl. lock modes can be added based on requirement



	IS	S	IX	SIX	X
IS	✓	✓	✓	✓	✗
S	✓	✓	✗	✗	✗
IX	✓	✗	✓	✗	✗
SIX	✓	✗	✗	✗	✗
X	✗	✗	✗	✗	✗

→ Problems that can occur:

→ Garbage values
 → Use short write locks (latches)
 The locks are for a few nanoseconds
 → Deg. 0 consistency

T ₁	T ₂
w(A=10)	w(A=25)
At the end, A may be neither 10 nor 25	



→ Last updates
 → Use long write locks
 → Degree 1 consistency

T ₁	T ₂
r(A)	w(A)
w(A) (commit)	
* T ₂ 's update is lost	

→ Dirty read

→ Use long write locks & short read locks

→ Deg. 2 consistency

T ₁	T ₂
w(A)	r(A)
abort	
* T ₂ read a wrong value	

→ Inconsistent read
 → Long write & long read locks

T ₁	T ₂
	r(A)
w(A)	
commit	
	r(A)

→ Phantom triple

Initial: A₁ = 1
 A₃ = 8



T ₁	T ₂
	Sum(A ₁ :A ₃) 9
Insert(A ₂ = 3)	
Commit	
	Avg(A ₁ :A ₃) 4 (4.5 expected)

→ Special locks for Phantoms:

→ Predicate lock (semantic)

- Efficient for simple predicates
- NP-hard "arbitrary"

→ Range lock

→ Tune range, lock on B⁺ tree index

→ Deadlock prevention: Simple soln. - Use timeout! (May result in unnecessary aborts)

Recovery Manager

→ Atomicity & Durability

→ Design Issues

→ Disk : In-place changes, or create a shadow copy?

→ Memory

→ FORCE: Force writing to disk @ commit time

→ STEAL: Allow buffer frames of uncommitted transactions to be 'stolen' by other transactions

	Advantage	Disadvantage
SHADOW	Shadow updates provide atomicity & instant recoverability	→ Sorted Trans (near each other) won't be preserved. → May lead to fragmentation
FORCE	Provides durability	→ Hard disk is slow, thereby resulting in poor response time.
STEAL	" "	→ Starvation may occur, reducing concurrency.

→ High-performance solutions:

→ In-place : Use logging to keep track

→ No-FORCE: Record new value of data @ commit in case of re-do

→ No-STEAL: " old " " " " from steal " " undo

→ Log

→ Stored in separate disk; writing will be faster because no competition & disk head can be kept in write posn.

→ Write-Ahead Logging (WAL) // intentions first, action later

→ Must force log record before writing data pg. // Provides atomicity

→ " " all log " for a transaction @ commit // " durability

→ Recovery protocol

→ Procedure

→ Analysis : Find 'winners' & 'losers'.

→ Undo : bet to 'safe' state

→ Redo : " " new 'safe' state

→ Assumptions :

→ Pg. level logging

→ Simple log types (SEX only)

→ Physical logging (before & after image only)

→ Idempotency of undo & redo

→ Increasing concurrency

→ Support operation logging (Logical logging)

→ " fancy log modes (IS, IX, SIX, INCR, DECR, etc)

→ " finer logs (record-level logging etc.)

idempotency is not preserved

value of a data item may reflect multiple transactions



implementation log record
ARIES

→ keeps CLR, follows redo-all (redo all transactions & undo losers only).

→ Each log record has a unique LSN which is given in increasing order.

→ " data pg (on disk) contains a page LSN to indicate who & when the last update was made.

→ flushed LSN : Last log record that was flushed. // Ensure page LSN $\leq$ flushed LSN for WAL

→ log record fields:

XID	transaction ID
type	
pageID	page which was updated
length	length of data object written (4 for int, etc.)
offset	offset " " within the pg.
before-image	
after-image	
prev LSN	used to sequence log records of a particular transaction, not required for correctness
undo next LSN	for CLR only

→ Types of log records:

Type	Meaning
Update	
Prepared	For distributed DBs only
Commit	
Abort	Not mandatory: absence of commit is abort in of itself
End	Transaction update has been written to disk: used to improve performance
CLR	Log records written during recovery to ensure idempotency.
Checkpoint	

Basically keeps track of how much 'cleanup' is done

→ Compensation Log Records

- Redo-only records of the undo operation of aborted transactions
- Permits operation logging, pg-oriented redo (for efficient recovery) & logical undo (for high concurrency in regular operations)

→ Transaction Table (TT)

- Used for concurrency
- One entry per transaction:
 - XID
 - status (aborted/committed/...)
 - last LSN (latest log record)
 - Undo next LSN (LSN of next log record to be undone)

→ Dirty Pg. Table (DPT) : Meant for REDO pass

- Page ID  on a rec LSN
- first LSN (first log entry to dirty the pg) // indicates earliest log record which have to be undone

→ Checkpointing

- Used to save time during recovery
 - begin-checkpoint : Indicates when checkpoint began
 - end-checkpoint : Contains TT & DPT
- TT & DPT are up-to-date as of begin-checkpoint timestamp
- Makes no attempt to flush dirty pgs. to disk. Hence effectiveness of checkpoint is limited by last pg. written to disk. Therefore it is optimal to flush dirty pgs. periodically.
- LSN of begin-checkpoint is stored in master record

→ Normal execution of Transaction

- Strict 2PL is maintained with STEAL, NO-FORCE
- When is a "transaction completed" msg. can be given to the user?

We need to verify this has been done using synchronous calls.

- When the log is flushed
 - * Data pg. need not have been flushed yet.
 - * This is to ensure lower response time.

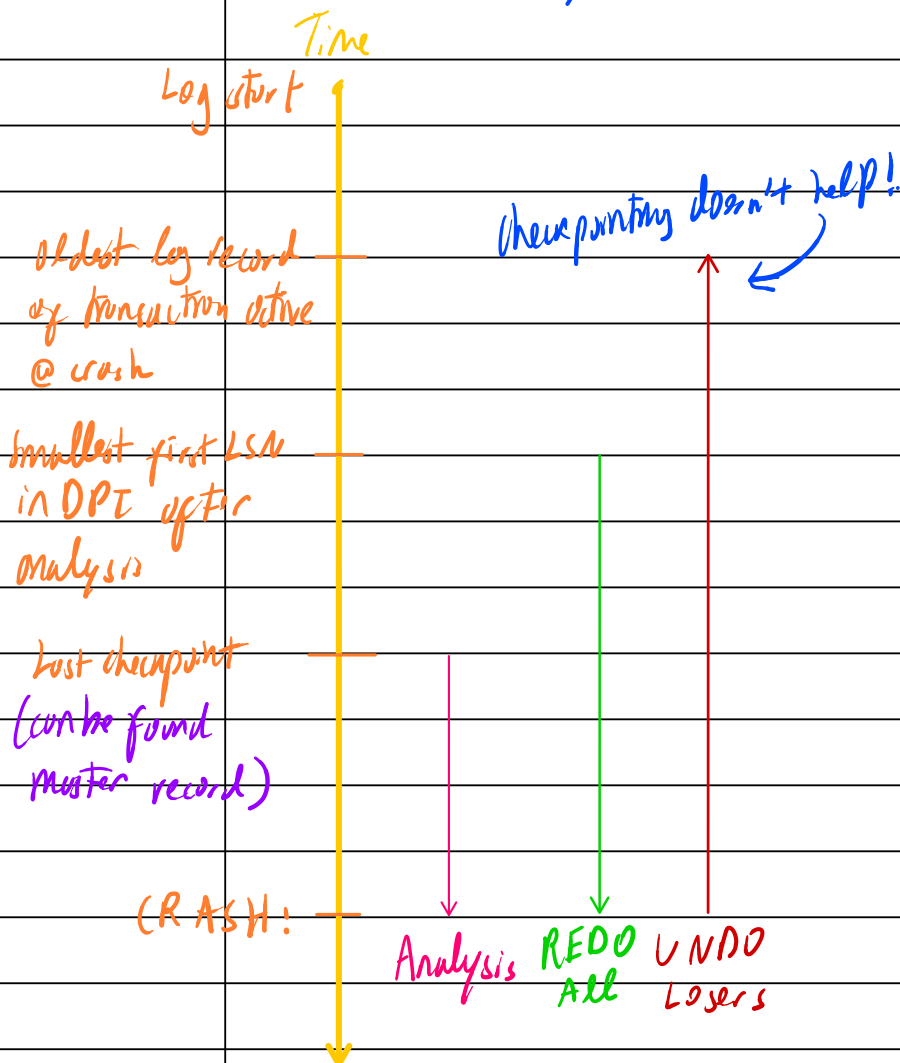
→ Transaction Commit

- Write commit record to log. // in RAM, to log 'tail'
- Flush all log records up to transaction's last LSN // to disc
- Once this is done, write commit log record. // in RAM

→ Transaction Abort (w/o crash)

- Write abort log record
- Replay the log in reverse order & UNDO the changes made to disc as part of STEAL.
this can be done by getting last LSN from TTE following back using prev LSN
we need to have a lock on data
- Before restoring prev. values, write a CLR
ensure logging is done during recovery also
- The victim field "undo next LSN" will have next gp. to be undone. // This is just prev LSN of the aborted transaction

→ Crash Recovery



→ Analysis Phase

→ Reconstruct state @ checkpoint (using end-checkpoint)

→ Scan log forward from begin-checkpoint

→ END record: Remove transaction from TT

→ Other records: Add transaction to TT (if not already present)

→ Set last LSN = LSN, change transaction records for control records

→ If UPDATE transaction if pg. P is not in DPT, add it to DPT & set first LSN = LSN.

→ TT will be accurate as of crash.

→ DPT " also be fairly accurate

→ May contain some 'false positives', i.e., pgs. which have been written to disk.

→ This problem can be solved by adding 'end-recon' log entry. However, this would create thousands of log entries which are useless except in use of a crash.

→ REDO phase

→ Scan forward from smallest first LSN in DPT.

→ For each record log, REDO the action. Unless:

Some other transaction must have flushed this dirty pg. → Affected pg. is not in DPT

→ " " " in DPT, but first LSN > LSN

My dirty pg. has been flushed, but someone → page LSN = LSN

came and dirtied it again

↖ needs to be read from DB disk, hence more expensive

can be given by IT

Repeat:

11 Prev. recovery fully cleaned up, but crashed before it could complete

If the LSN is a LLR and indonent LSN == NULL:
Write end record for this transaction

11 Prev. recovery halfway cleaned up, but crashed before it could complete

If the LSN is a CLR and $\text{undo_next_LSN} \neq \text{NULL}$:
Add undo_next_LSN to to_undo

If the LSN is a update

Undo the update

Write the CLR

Add prevLSN to to_undo

If the LSN is a abort / prepare

Add prevLSN to to_undo

Until to-undo is not empty.

→ Addl. crash issues

- If system crashes during analysis, restart analysis phase again
- REDO:

$\rightarrow$ h n u x o REÇO:

→ We will get more false +ve in DPT; hence little more expensive

→ Some REDO's need not be done since $pageLSN =$

This can be done using dynamic workload management

update record LSN // redo has already been done for this LSN

→ Max. loss for REDD can be minimised by flushing frequently.

$\rightarrow$ transactions UNDO overlapping long-running

Memory Management

→ Why LRU?

- Easy to implement
- Temporal locality

→ Why not LRU?

- Does not understand reference patterns
- Does not distinguish b/t index & data pgs.

→ Prior solutions

- Create two buffer "pools" for index & data. Needs to be done manually. Hence, it is expensive and not dynamic.

→ Query Plan analysis

- Query optimiser should provide hints on the usage pattern of a query plan.
- However, this may get complicated for multi-user systems.

→ LRU-n

- Maintains history of past K references, hence discriminates b/t pg. sets w/lt difference reference patterns.
- Evolves to access patterns and self-referent (does not require add'l. hints)
- Small overheads
- Small K for stable access patterns, large K for stable data access patterns.

→ Independent reference patterns: (Used in most systems, i.e. hard disks)

- References to objects are random variables (independent)

→ A: "Ideal" algorithm (as in GATE SE-OS)

→ Backward n -distance: The distance from current to the n^{th} last reference of a pg.

→ If a pg has $< n$ references, its backward n -distance is ∞ .

→ The pg. with largest backward n -distance is the victim pg.

→ Standard LRU is used as tie-breaker.

→ Problems:

→ Early pg. replacement: There are 4 ways in which the same pg. can be accessed:

→ Intra-transaction: Read record on pg., then update record.

→ Transaction-Retry: A transaction aborts, then restarts, then attempts to access the same pg.

→ Intra-process: Access initial records on pg., then commit

→ Inter-process: Independent access by multiple transactions.

→ Solution: Interarrival times must be non-correlated. Each successive access on the same pg. within a time period (correlated reference period) are taken to be unrelated.

↖ 5 seconds

→ Retained information

→ Each time a pg. is referenced, it is put into the buffer.

→ If $b_i(p, n) = \infty$, it is the most prime candidate for victim place.

→ What if the pg. is popular but it's just not referenced enough times?

→ Solution: We need to maintain history about a pg. for some period (Retained information period (RIP)) after its most recent access.

2002s

→ Additional data structures we used:

→ $HIST(p)$: History control block of pg p

→ Contains times of 2 most recent (uncorrelated) references of p .

→ $LAST(p)$: Most recent reference to p

→ These data structures are stored in memory, rather on disk to prevent dirty pg. write back everytime a pg. is accessed.

→ Maintained for all pgs. with bookmark $k < R/P$

→ Optimal value of $k = 2$. $k = 2$ provides a huge improvement over $k = 1$, whereas the improvement from $k = 2$ to $k = 3$ is minor.